

Counter-Strike Polymarket Betting System — V2:

Alpha Expansion

Audience: an AI engineering agent extending the v1 system. **Prerequisite:**

`cs_betting_system_spec.md` (v1) is built, deployed, and has demonstrated ≥ 60 days of **positive CLV with rolling ECE $< 3\%$** in paper trading or low-stake live trading. **Status:** v2 spec. This document layers additional alpha sources onto the v1 foundation. It does **not** replace any v1 component.

1. Context & What V2 Adds

V1 delivered:

- A calibrated map-winner model.
- A veto + Monte Carlo aggregator producing match-winner, map-handicap, total-maps, correct-score probabilities.
- A round-level simulator.
- Polymarket ingestion, edge filter, fractional Kelly sizing.
- Walk-forward backtesting and monitoring.

V1 hunts edge in the headline market and a few obvious derivatives, sized conservatively.

V2 expands the surface area in three directions:

1. **More markets:** price every derivative the simulator can price; arbitrage against sharper books.
2. **More signals:** feed the model with structural features the market under-weights (AWPer form, fatigue, LAN/online splits, veto fingerprints, tournament incentives).
3. **Faster reactions:** pipelines that re-price within seconds when news drops, when vetoes are published, when a map ends mid-match.

Each alpha is built as an **isolated module** with its own switch, its own capital allocation, its own metrics, and its own kill switch. Nothing in v2 may degrade v1's baseline.

2. Design Principles for V2

- 1. **Modular alpha.** Each strategy is a plugin: feature contributions, market-pricing function, edge filter, capital cap. Toggle on/off independently.
- 2. **Capital partitioning.** Allocate bankroll fractions across strategies. A blow-up in one strategy can't drain the others.
- 3. **Per-strategy metrics.** Every strategy logs its own CLV, ROI, calibration. A strategy that drifts negative gets auto-disabled.
- 4. **Alpha decay is real.** Track ROI and CLV trends per strategy with a 90-day window. Assume any edge halves every 12–24 months.
- 5. **Speed is a strategy.** Some alphas exist *only* below a latency threshold. Treat latency as a first-class product metric.
- 6. **Bookmaker risk.** Winning against soft books gets you limited. Cross-book strategies need account hygiene (stake sizing, bet variety, multiple accounts where legal).
- 7. **Never trade what you can't size into.** Soft prices on illiquid markets are paper alpha. Always check depth.

3. Strategy Catalog Overview

#	Strategy	Build Cost	Expected Edge	Latency Need	Phase
A1	Cross-book arbitrage (vs Pinnacle)	Low	1–3% per bet, high frequency	Minutes	A
A2	Coherent derivative pricing	Low (reuses v1 sim)	2–5% per bet, medium frequency	Minutes	A
B1	News-driven fast reactions	High	5–15% per bet, low frequency	<60 sec	B
B2	Post-map live repricing (BO3/BO5)	Medium	3–8% per bet, medium frequency	<2 min	B
B3	Veto-conditional repricing	Medium	2–6% per bet	<30 sec	B

C1	Veto fingerprint model	Medium	Tightens A2 by ~1%	None	C
C2	AWPer-specific modeling	Medium	1–3% on AWP-heavy teams	None	C
C3	Map-pool asymmetry / decider modeling	Medium	Tightens totals & correct-score	None	C
C4	Online vs LAN regime split	Low	1–3% on regime-sensitive teams	None	C
C5	Travel & time-zone effects	Low	0.5–2% on cross-region matches	None	C
C6	Schedule fatigue	Low	0.5–2% in dense formats	None	C
D1	Roster honeymoon curve	Low	1–3% on first 10 matches of new lineup	None	D
D2	Coach vs player change differential	Low	0.5–1%	None	D
D3	Patch/meta shift detection	Medium	Episodic, large after major patches	None	D
D4	Tournament incentive modeling	Medium	1–3% on incentive-asymmetric matches	None	D
D5	Public bias / contrarian signal	Low	0.5–2% on public favorites	None	D
D6	Pistol-round specialist model	Medium	2–5% on pistol markets when offered	None	D

Expected-edge figures are **planning estimates**, not promises. Validate empirically before sizing up.

4. Phase A — Quick Wins

These two strategies should be built first. A1 doesn't need a model and validates the pricing pipeline. A2 reuses everything v1 already built.

4.1 Cross-book Arbitrage (A1)

Thesis. Pinnacle is the sharpest CS book in the world. Its devigged prices are the closest thing to a market-truth signal. When Polymarket prices an outcome more than ~3 percentage points off devigged Pinnacle, the Polymarket price is usually wrong. This is the lowest-skill, highest-frequency edge available.

Inputs.

- Pinnacle odds (REST API if you have access; scrape via a third-party odds aggregator otherwise — OddsPortal, OddsJam, BetBurger).
- Bet365, GG.BET, 1xBet as secondary references.
- Polymarket order book (already ingested in v1).

Devig method. For a two-way market with prices p_a and p_b (each between 0 and 1):

```
def devig_power(p_a, p_b, k_init=1.0):
    """Power-method devig. Solves for k such that  $p_a^k + p_b^k = 1$ ."""
    from scipy.optimize import brentq
    f = lambda k: p_a**k + p_b**k - 1.0
    k = brentq(f, 0.5, 2.0)
    return p_a**k, p_b**k
```

Use power method or Shin method; both outperform the simpler “multiplicative” devig. Document the choice; never mix methods across strategies.

Schema additions.

```
CREATE TABLE external_books (
    book_id      SMALLSERIAL PRIMARY KEY,
    name         TEXT UNIQUE,          -- pinnacle, bet365, ggbet, ...
    is_sharp     BOOLEAN,
    source       TEXT                  -- api, scraper, aggregator
);

CREATE TABLE external_odds_snapshots (
    snapshot_id  BIGSERIAL PRIMARY KEY,
    book_id      SMALLINT REFERENCES external_books,
    match_id     BIGINT REFERENCES matches,    -- nullable until linked
    market_type  TEXT,                      -- match_winner, map_handicap, total_maps
    taken_at     TIMESTAMPTZ NOT NULL,
    outcome      TEXT,
    price_decimal NUMERIC,                  -- decimal odds, e.g. 1.85
    implied_raw  NUMERIC,
    implied_devig NUMERIC
);
```

```
CREATE INDEX ON external_odds_snapshots (match_id, market_type, taken_at DESC);
```

Match-and-market linking. The hardest engineering problem in A1 isn't pricing — it's resolving "Pinnacle's match" to "Polymarket's match" to "our `matches.match_id`".

Implement an entity resolver:

```
def resolve_match(book_match: dict) -> int | None:
    # Try in order:
    # 1. (event_id_known, team1_name, team2_name, kickoff ± 2h) -> exact
    # 2. (team1_alias, team2_alias, kickoff ± 6h) -> alias map
    # 3. (date, team1_country, team2_country, BO) -> heuristic
    # 4. Flag for human review.
    ...
```

Maintain a manual-override table for stubborn cases. Audit weekly.

Edge filter (A1).

```
def crossbook_edge(pm_ask: float, pinnacle_devig: float,
                  spread_threshold: float = 0.03):
    edge = pinnacle_devig - pm_ask
    return edge if edge > spread_threshold else 0.0
```

Threshold starts at 3 percentage points; tighten or loosen by observed CLV after 100 bets.

Sizing. Quarter Kelly on $\text{edge} / (1 - \text{pm_ask})$, capped at 1% bankroll per bet (lower than v1's 2% — A1 trades frequency for size).

Operational notes.

- Snapshot Pinnacle at least every 60 seconds; faster around match start.
- Always trade on the *current* Pinnacle price, not a stale snapshot. The whole strategy fails if your reference is more than 2 minutes old.
- Pinnacle moves fast. If their price has just moved sharply, wait one cycle before betting — you might be late to the same news.

Metrics. Per-week: number of opportunities found, number bet, average edge taken, realized ROI, CLV (vs Pinnacle close), realized vs expected variance.

4.2 Coherent Derivative Pricing (A2)

Thesis. Most books price match-winner, handicap, totals, and correct-score independently

using internal formulas. The v1 simulator produces a coherent joint distribution across all of these. Where the market's marginals contradict each other (or contradict the simulator), edge exists.

What v2 adds to v1. v1 already prices these markets. v2:

1. **Scans every market on every match**, not just match-winner.
2. **Cross-checks for internal contradictions** in the market.
3. **Ranks opportunities** by edge $\times$ confidence $\times$ liquidity, surfaces the top N.

Implementation.

```
def price_all_markets(match: Match, sim_output: SimResult) -> dict[str, float]:
    return {
        "match_winner_team1":      sim_output.p_team1_match,
        "match_winner_team2":      1 - sim_output.p_team1_match,
        "map_handicap_-1.5_team1": sim_output.p_2_0_team1,
        "map_handicap_+1.5_team1": 1 - sim_output.p_2_0_team2,
        "total_maps_over_2.5":      sim_output.p_3_maps_played,
        "total_maps_under_2.5":     1 - sim_output.p_3_maps_played,
        "correct_score_2_0":         sim_output.p_2_0_team1,
        "correct_score_2_1":         sim_output.p_2_1_team1,
        "correct_score_1_2":         sim_output.p_1_2_team1,
        "correct_score_0_2":         sim_output.p_0_2_team1,
        # Map-level markets
        **{f"map{i}_winner_team1": p for i, p in enumerate(sim_output.per_map_pro
        # Round-level (from round simulator)
        **{f"map{i}_total_rounds_over_24.5": p
            for i, p in enumerate(sim_output.per_map_round_totals)},
    }
```

Internal contradiction check. A market is internally inconsistent when its marginals don't agree under any joint distribution. Example: if $P(2-0 \text{ team1}) + P(2-1 \text{ team1}) > P(\text{match_winner team1})$ by more than vig, something is broken. When this happens, trade the side(s) that align with the simulator.

```
def internal_consistency_violations(market_prices: dict[str, float]) -> list[str]:
    violations = []
    # match-winner vs correct-score
    cs_team1 = market_prices["correct_score_2_0"] + market_prices["correct_score_2_1"]
    if cs_team1 > market_prices["match_winner_team1"] + 0.02:
        violations.append("correct_score sum exceeds match_winner")
    # handicap vs correct-score
    if abs(market_prices["map_handicap_-1.5_team1"]
```

```

        - market_prices["correct_score_2_0"]) > 0.015:
    violations.append("handicap-2.0 ≠ correct-score 2-0")
    return violations

```

Output. A ranked opportunity table:

```

| match_id | market | model_p | market_p | edge | depth_$ | confidence |
recommended_stake |

```

Sort by `edge × log(depth) × confidence` . Show top 20 each scan cycle.

Confidence score. Heuristic in [0, 1]:

- +0.3 if both teams have >50 matches in the last 6 months.
- +0.2 if no roster changes in last 30 days for either team.
- +0.2 if event is LAN tier-S (the model is best-validated there).
- +0.15 if the predicted map veto matches historical patterns (see C1).
- +0.15 if no flagged news event in the last 24h.

Use confidence to **scale the edge threshold**, not the stake. A 4% edge at confidence 0.9 acts; a 4% edge at confidence 0.4 does not.

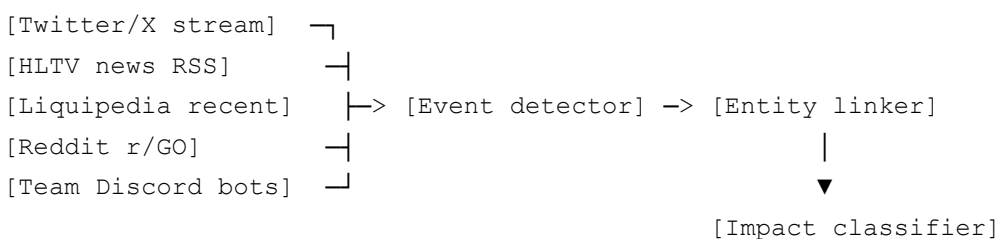
5. Phase B — Live Reaction Systems

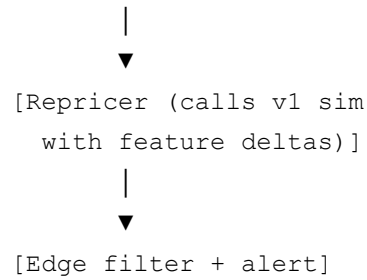
These strategies trade information speed for edge size. They require new streaming infrastructure.

5.1 News-Driven Fast Reactions (B1)

Thesis. Material events (stand-ins, illness, visa issues, last-minute roster changes) shift true win probability by 5–15 percentage points. Polymarket lags by 5–60 minutes on these. A pipeline that detects, classifies, and re-prices in under 60 seconds captures the gap.

Pipeline.





Sources to monitor.

- Twitter/X streaming API: curated list of ~300 accounts (pro players, team accounts, journalists like Volamel, Slasher, dust2_us, Dexerto.com, HLTVconfirmed hosts, team CEOs).
- HLTV news RSS: parse every new article; entity-extract teams and players.
- Liquipedia recent changes: roster move announcements.
- Reddit r/GlobalOffensive: high-upvote-velocity threads as a noisy signal.
- (Optional) Beat-reporter Discord webhooks where access is available.

Event taxonomy. Pre-classify into a fixed set:

Event	Typical Impact (P pts on map)	Direction
Stand-in for star player	-8 to -15	Affected team
Stand-in for role player	-3 to -6	Affected team
Coach playing as stand-in	-10 to -18	Affected team
Confirmed illness, still playing	-2 to -5	Affected team
Visa / travel delay, playing remote at LAN	-4 to -8	Affected team
Internal conflict / public dispute	-1 to -4	Affected team
Roster move announced (pre-event)	varies; defer to roster model	Both
Equipment / monitor incident at LAN	-1 to -2	Affected team, transient
Practice partner ghosting	-0.5 to -1	Affected team

These are priors learned from history (mine them from past matches where a known event occurred and compare actual vs pre-event predicted). They are not fixed constants.

Tech stack.

- **Stream consumer:** Python `tweepy` for Twitter (with v2 filtered stream), `feedparser` for RSS, `praw` for Reddit.
- **Queue:** Redis Streams or Redpanda. Latency budget ~5s end-to-end before classification.
- **NER:** spaCy with custom entity model for players/teams (train on HLTV article corpus). Fallback to an LLM call for ambiguous cases.
- **Classifier:** fine-tuned small LM (e.g., distilbert) on a few hundred hand-labeled events. Outputs event type + confidence.
- **Repricer:** takes (match_id, event_type, affected_entity, magnitude), applies feature delta, re-runs the simulator, emits new market prices.
- **Alert:** Slack/Telegram with edge details and one-click acknowledgement.

Latency budget (target).

Stage	Budget
Source → queue	5 sec
NER + entity link	2 sec
Event classify	3 sec
Repricing	5 sec
Edge calc + alert	1 sec
Total	16 sec

Risk: false positives. A misclassified event = bad bets. Mitigations:

- Require confidence > 0.85 from the classifier.
- Cross-confirm across 2+ sources within 5 minutes for high-impact events.
- Hard cap stake at 0.5% bankroll for news-driven bets until the pipeline has 100 confirmed-correct detections.
- Maintain a "quiet list" of accounts known for jokes/misinformation.

Schema.

```
CREATE TABLE news_events (  
  event_id      BIGSERIAL PRIMARY KEY,  
  detected_at   TIMESTAMPTZ NOT NULL,  
  source        TEXT,           -- twitter, hltv, liquipedia, reddit  
  source_url    TEXT,  
  raw_text      TEXT,  
  event_type    TEXT,           -- stand_in_star, illness, ...  
  confidence    NUMERIC,  
  team_id       BIGINT REFERENCES teams,  
  player_id     BIGINT REFERENCES players,  
  match_id      BIGINT REFERENCES matches,  
  impact_delta  NUMERIC,        -- estimated P-point shift on affected team  
  was_correct   BOOLEAN,        -- backfilled after match  
  notes         TEXT  
);
```

Every detected event is logged, whether or not it triggered a bet. The `was_correct` field is reviewed weekly to retrain the classifier and recalibrate impact deltas.

5.2 Post-Map Live Repricing in BO3/BO5 (B2)

Thesis. After map 1 ends in a BO3, the conditional $P(\text{match winner} \mid \text{map 1 result})$ changes substantially. Polymarket sometimes keeps the match market open during the break between maps. For 5–15 minutes after map 1 closes, prices lag the new conditional.

Pipeline.

1. Detect map end (HLTV live API, or scrape match page every 30s during live matches).
2. Update simulator state with realized map 1 result.
3. Re-run the aggregator conditioning on remaining maps in the BO3 set.
4. Diff against current Polymarket price.
5. Alert if edge > threshold and market is still open.

Subtlety: momentum. Empirically, teams that just lost map 1 are *slightly underpriced* on map 2 — markets overweight momentum. From v1 demo data, compute the historical $P(\text{team wins map 2} \mid \text{lost map 1})$ for each team and bake it into the conditional.

```
def post_map_conditional(match, played_maps, remaining_maps, base_probs):  
    # base_probs[m] = pre-match P(team1 wins map m)
```

```
# played_maps = [(map_name, winner)]
momentum_adj = compute_momentum_adjustment(match, played_maps)
# momentum_adj is small, in [-0.03, +0.03]; learned from history.
adjusted = {m: clip(base_probs[m] + momentum_adj.get(m, 0), 0.05, 0.95)
            for m in remaining_maps}
return simulate_match_from_state(played_maps, remaining_maps, adjusted)
```

Latency target. Map-end detection within 60 seconds. Repricing within 5 seconds of detection. Alert within 10 seconds of repricing.

Cap on B2 bets. Same per-match cap as v1 (4% bankroll). A bet placed pre-match plus a B2 bet on the same match shares this cap — they're highly correlated.

5.3 Veto-Conditional Repricing (B3)

Thesis. True match-winner probability shifts 3–8 percentage points between vetoes published and vetoes complete. Markets that re-price slowly during this window are exploitable.

Veto sources.

- HLTV match page (vetoes posted live, sometimes minutes before map start).
- Tournament-operator overlays/streams (manual operator).
- Twitter accounts that post vetoes (e.g., @CSGO_veto_tracker style).

Implementation.

1. Poll veto sources every 15 seconds for live matches.
2. On each new veto step, update the `veto_actions` table and re-simulate with the partial veto state.
3. Diff against Polymarket. Most edge appears at:
 - After both teams' first bans (the favorite's strongest maps are off).
 - After picks (when the played-map set is locked).

Caveat. Polymarket sometimes pauses or resolves the veto-relevant market before vetoes are published, especially for non-marquee matches. B3 only applies when the market stays open through vetoes.

6. Phase C — Structural Alpha

These strategies modify the model rather than running parallel pipelines. They're integrated into v1's training and feature pipeline.

6.1 Veto Fingerprint Model (C1)

Thesis. Teams have stereotyped veto patterns. Modeling them sharpens the played-map distribution, which sharpens total-maps and correct-score pricing.

Model. Per veto step s , a multinomial classifier:

```
P(team_t bans/picks map_m | step s, both teams' map strengths, recent veto history, head-to-head veto history)
```

Features.

- Team's per-map Glicko rating (own).
- Opponent's per-map rating.
- Team's historical veto frequencies at this step over last 90 days.
- Head-to-head veto history (last 5 meetings).
- Event tier (teams may behave differently in tier-S vs tier-B).
- BO format.

Training data. Every historical veto action is a labeled example. Sample size is enormous (tens of thousands of actions over a few years).

Validation. Per-step top-1 and top-2 accuracy vs a baseline (always-most-frequent). Improvement of 5+ points over baseline is the bar.

Integration. Replace v1's naive veto sampler with the fingerprint model in the Monte Carlo aggregator. Expect total-maps and correct-score calibration to improve by ~0.5–1 point ECE.

6.2 AWPPer-Specific Modeling (C2)

Thesis. Map outcomes correlate strongly with the day-to-day form of the team's primary AWPPer. Treating all five players equivalently in team aggregates loses information.

Identifying the primary AWPPer. Per (team, map, time-window), the player with the highest AWP-kill share. Recompute every 30 days.

AWPPer-specific features.

- AWPPer rating 2.0 rolling 30 / 90 days.
- AWPPer opening-kill rate, recent.
- AWPPer multi-kill rate (3K+).
- AWPPer impact on rounds (round-win rate when AWPPer survives vs dies).
- AWPPer variance (std dev of per-map rating across last 20 maps) — high variance = boom-or-bust.

Integration. Add as features to the map-winner model. Expect 1–3 point improvement on matches involving AWP-heavy teams (NAVI, Vitality, MOUZ historically).

6.3 Map-Pool Asymmetry & Decider Modeling (C3)

Thesis. The decider map in a BO3 is biased toward the weaker team's comfort, because both teams ban the favorite's strong maps. Naive models that pick decider as "random across remaining maps" miss this.

Approach. Compute `P(map m is decider | veto state, both teams' map preferences)` directly from C1. Feed into the aggregator's joint distribution over played-map sets.

Effect. Underdogs in BO3 are slightly underpriced on map handicap +1.5 (they reach map 3 more often than naive models predict, and they win it more often than naive models predict). Magnitude: ~0.5–1.5 percentage points on handicap markets.

6.4 Online vs LAN Regime Split (C4)

Thesis. Some teams perform measurably differently online vs LAN. v1 includes `is_lan` as a feature but doesn't model the team-specific interaction.

Implementation.

- Compute two ratings per team: `rating_online` and `rating_lan`, with their own rolling windows.
- Add interaction feature `(is_lan, team_id)` via target encoding with regularization.
- Cross-validate: does the model with the split outperform without it?

Risk. Tier-1 teams play almost exclusively LAN now (post-2022). Online data is increasingly thin and dated. Be conservative on LAN-vs-online inferences for top teams; weight more aggressively for tier-2.

6.5 Travel & Time-Zone Effects (C5)

Thesis. Cross-region travel degrades day-1 performance. Documented in traditional sports; observable in CS for Asia↔EU↔NA crossings.

Features.

- Geographic distance traveled in last 7 days (Haversine on event venue → team home base).
- Time zone delta from team home.
- Days since arrival at event city (proxy: days since first match at the event).
- Specific “long-haul” flag: distance > 5000 km or TZ delta > 6h.

Modeling. Add as features. Optionally fit a dedicated “travel adjustment” term that decays over the first 3 days of the event.

Magnitude. Typically -0.5 to -2 percentage points on day-1 win probability for long-haul teams. Higher on map 1 of the day.

6.6 Schedule Fatigue (C6)

Thesis. Aim degrades measurably under fatigue. Visible in demo data: lower headshot %, slower reaction-time proxies, more eco-round losses in late matches of long days.

Features.

- Matches played in last 24h / 12h / 4h.
- Maps played in last 24h.
- Rounds played in last 24h (better proxy than maps; some maps are short).
- Days since last rest day.

Modeling. Add as features. Expect strongest effects in Swiss formats with same-day rematches and in long playoff days.

Magnitude. -1 to -3 percentage points per additional match played that day. Larger for the team that’s been on stage longer (the team coming off a BO3 vs a team that just arrived).

7. Phase D — Subtle Effects

Smaller individually but stack. Build after Phase C has shown measurable lift.

7.1 Roster Honeymoon Curve (D1)

Thesis. New rosters outperform their pre-change Elo for ~10 matches (motivation, opponent unfamiliarity), regress, then climb steadily as systems gel.

Feature. `matches_with_current_roster` — count of matches since the current 5-man lineup was assembled.

Modeling. Bucket: [0–5, 6–10, 11–25, 26–50, 51+]. Or fit a continuous spline. Expect a measurable U-shape.

Use. Particularly relevant for futures and event-winner markets, where the roster's trajectory over a multi-week event matters more than a single match.

7.2 Coach vs Player Change Differential (D2)

Thesis. A coach change with same five players shifts win rate less than markets imply. A player change shifts it more. Markets often treat them similarly.

Features.

- `days_since_coach_change`, `days_since_player_change` (separately).
- `change_type` categorical: coach-only, role-player swap, star swap, IGL swap.

Empirical priors (validate on your data):

- Coach change: ~30–50% of the impact of a player change.
- IGL swap: highest impact, ~1.5x of an average player swap.

7.3 Patch / Meta Shift Detection (D3)

Thesis. Major patches (map reworks, weapon balance, smoke physics) shift the CT/T balance and team-specific strengths. Markets take 2–6 weeks to adjust. Your model needs to adjust faster.

Implementation.

- Maintain a `patches` table with date and a free-text summary.
- For each patch, define `affected_scope`: global, specific maps, specific weapon classes.
- Weight pre-patch data with exponential decay when the patch affects relevant features.

Detection of unobserved shifts. Weekly SHAP-value monitoring on the production model. If the importance of a feature shifts > 25% week-over-week, alert. This catches meta shifts you didn't manually flag.

Re-rating after major patches. Re-fit per-map Glicko ratings with a higher prior variance

for 30 days after a map rework on that specific map. The model should be appropriately uncertain immediately post-patch.

7.4 Tournament Incentive Modeling (D4)

Thesis. Teams play differently when standings make a match must-win, dead-rubber, or tank-for-seeding. Markets often miss this.

State tracking. Per (team, event), maintain live standings: matches played, wins, losses, current placement, mathematical scenarios.

Derived features per match.

- `is_elimination_match` for either team (loss = out).
- `is_dead_rubber` (both teams already qualified or both already eliminated).
- `seeding_incentive` (qualifying vs avoiding a top seed in the next stage).
- `must_win_for_advancement` (win = qualify, loss = elimination).

Modeling. Add as features. Expect:

- Dead-rubber matches: variance up, favorite edge slightly reduced. Markets often don't adjust.
- Elimination matches: favorite edge slightly increased (better teams clutch up).

Caveat. Hard to validate cleanly with limited data. Treat impact estimates as low-confidence until you've seen 200+ matches per incentive bucket.

7.5 Public Bias / Contrarian Signal (D5)

Thesis. "Public" teams (FaZe, NAVI, G2, Liquid, big-org rosters generally) attract recreational money on Polymarket regardless of form. When in a slump, their prices stay artificially short. Fading slumping public favorites is a documented edge in traditional sports markets.

Public-team indicator. Top-N teams by historical Polymarket trading volume over the last 90 days. Update monthly.

Slump indicator. Team Elo movement < -50 over last 30 days *and* current Elo below 90-day moving average.

Strategy. When a public team in a slump is priced ≥ 3 percentage points above devigged Pinnacle, treat the gap as actionable (assuming Pinnacle agrees with our model).

Magnitude. Small per-bet edge but persistent and frequent. Plays well as a sub-strategy of

A1.

7.6 Pistol-Round Specialist Model (D6)

Thesis. Pistol rounds are higher-variance than gun rounds but not as random as markets price them. Some teams have measurable pistol edges, often coaching-driven (early-round set plays).

Model. Dedicated binary classifier:

`P(team1 wins first pistol of map | map, teams, IGL form, recent pistol history)`

Features.

- Team’s pistol-round win rate over last 90 days, per map.
- Team’s pistol-round win rate as CT vs T (asymmetric).
- IGL’s pistol-round impact (rating in pistol rounds specifically).
- Opponent’s pistol-round defense.
- Map (pistol balance varies dramatically: Mirage is roughly 50/50, Inferno tilts CT-pistol).

Markets. Pistol-round handicap and first-pistol props appear sporadically on Polymarket and on the GG.BET side of the cross-book strategy. Edge can reach 5+ percentage points but volume is thin.

8. Capital Allocation Across Strategies

V1 uses one bankroll. V2 partitions it. A blow-up in one strategy must not drain the others.

Starting allocation (review monthly):

Bucket	Fraction	Notes
v1 core (match winner, basic derivatives)	40%	Proven baseline.
A1 cross-book arb	20%	Low edge, high frequency.
A2 coherent derivatives	15%	Reuses v1, scans broader.
B1 news reactions	5%	High edge, low frequency, high variance.
B2 post-map live	5%	Medium edge, opportunistic.

B3 veto-conditional	5%	Opportunistic.
Reserve	10%	Drawdown buffer; never deployed in normal operation.

Rebalancing rule. Every 30 days, rebalance toward the strategy with the highest 90-day risk-adjusted return (Sharpe-like: realized ROI / realized std-dev), subject to a max allocation of 35% per strategy. Never let any one strategy exceed 35% of bankroll.

Per-strategy kill switch. If a strategy's rolling 100-bet CLV goes negative for two consecutive weeks, freeze it pending review.

9. Alpha Decay Monitoring

Edges erode. Track explicitly.

Per-strategy decay dashboard.

- Rolling 30-day, 90-day, 180-day mean edge taken.
- Rolling CLV trend (linear fit slope; negative slope is a warning).
- Average market reaction time to news events (B1): if it's shrinking, B1 is decaying.
- Average price gap between Polymarket and Pinnacle (A1): if shrinking, A1 is decaying.

Decay response protocol.

- Decay > 25% YoY: review thresholds, possibly tighten edge filter.
- Decay > 50% YoY: deprecate or rebuild strategy.
- Sudden cliff (decay > 30% in 30 days): freeze strategy, investigate. Likely a structural change (new market participant, API change, model regression).

10. Infrastructure Changes from V1

New systems.

- **Streaming:** Redpanda or Redis Streams for low-latency event processing (B1, B2, B3).
- **Cross-book ingestion service:** dedicated process polling external books at 30–60s intervals.

- **NER service:** containerized spaCy + LLM fallback for B1.
- **Live match poller:** every 30s during live matches; HLTv live page + tournament APIs.
- **Strategy router:** decides which strategy a candidate bet belongs to, applies per-strategy caps and limits.

Upgraded systems.

- **Online feature store:** must serve features in < 100ms. Redis-backed Feast online store, pre-materialized for upcoming matches.
- **Monitoring:** per-strategy dashboards, latency histograms, decay tracking.
- **Bet log:** add `strategy_id`, `latency_ms` (from trigger event to alert), `triggered_by` (event_id or scheduled scan).

New repository layout (extends v1).

```

cs-bet/
├── sports/cs/
│   ├── ... (v1 dirs)
│   └── v2/
│       ├── crossbook/           # A1
│       ├── derivatives/        # A2 (scanner + consistency checker)
│       ├── news/               # B1
│       │   ├── stream/
│       │   ├── ner/
│       │   └── classifier/
│       ├── live/               # B2, B3
│       │   ├── map_end_poller.py
│       │   └── veto_poller.py
│       ├── features_v2/        # C1-C6
│       │   ├── awper.py
│       │   ├── travel.py
│       │   ├── fatigue.py
│       │   └── ...
│       └── effects/            # D1-D6
├── core/
│   ├── strategy_router/        # NEW
│   ├── capital_allocator/      # NEW
│   └── decay_monitor/          # NEW

```

11. Implementation Roadmap

Assumes v1 is in production. Each phase is roughly 4–6 weeks.

Phase A (weeks 1–6 of v2)

- Build external-book ingestion (Pinnacle + 2 backups).
- Build entity resolver for cross-book matching.
- Implement A1 cross-book strategy. Paper trade for 30 days.
- Build A2 derivatives scanner and consistency checker on top of v1 simulator.
- Live-test A1 and A2 with small caps (50% of allocated bankroll).

Phase B (weeks 7–14)

- Stand up streaming infrastructure (Redpanda/Redis Streams).
- Build news source ingestion (Twitter, HLTV, Liquipedia, Reddit).
- Train NER on player/team corpus.
- Train event classifier on hand-labeled events.
- Wire repricer; instrument latency end-to-end.
- B2: build live map-end poller, post-map repricer.
- B3: build veto poller, partial-state simulator.
- Paper trade B1/B2/B3 for 30 days. Critical: validate no false-positive bets get sized.

Phase C (weeks 15–22)

- C1 veto fingerprint model: train, validate, integrate into aggregator.
- C2 AWPPer features: extract from demos, add to map-winner model.
- C3 decider distribution: integrate into joint output.
- C4 online/LAN split: separate ratings, interaction features.
- C5 travel features.
- C6 fatigue features.
- Re-run walk-forward backtest against v1 baseline. Each addition must show measurable lift on either log loss or ECE.

Phase D (weeks 23–30)

- D1 honeymoon curve.
- D2 coach/player differential.
- D3 patch detection and post-patch rating adjustments.
- D4 tournament-incentive features.
- D5 public-bias indicator (integrates with A1).
- D6 pistol-round model.
- Final walk-forward, sealed hold-out validation, deployment.

12. Risks Specific to V2

Risk	Mitigation
Cross-book account limits	Distribute volume, vary stake sizes, multiple accounts where legal.
News false positives	High confidence threshold, cross-source confirmation, small per-bet cap, weekly classifier retraining.
Latency regressions in B1/B2	Per-stage latency SLOs, alert on degradation, automatic fallback to slower confidence-thresholded path.
Alpha decay accelerates with more participants	Monitoring per §9, willingness to deprecate strategies.
Polymarket API or rules change	Maintain a feature-flag for each Polymarket-specific assumption (fees, market types, resolution rules).
Streaming infra outage	Strategies B1/B2/B3 fail closed (no bets), not open (random bets). v1 baseline keeps running.
Strategy correlation: a single big news event triggers v1 + A2 + B1 bets on the same match	Strategy router enforces per-match aggregate cap across all strategies.
NER misidentifies a less-known player	Fallback to LLM-based entity resolution for unmatched mentions; review weekly.

Model overfitting from too many features	Feature ablation tests every two weeks; remove features that don't lift held-out metrics.
Operator burnout from alert volume	Tiered alerting: only top-N opportunities surfaced live; rest go to a digest.

13. Definition of Done for V2

Each strategy module is “done” when:

1. It has its own walk-forward backtest with documented metrics.
2. It has paper-traded for ≥ 30 days with positive CLV.
3. It exposes a feature flag to enable/disable without redeploy.
4. It writes to the unified bet log with `strategy_id` and `latency_ms`.
5. It has per-strategy dashboards and a kill switch wired to monitoring.
6. It has documented expected edge, observed edge, and a decay-monitoring threshold.

V2 as a whole is “done” when:

- All Phase A and B strategies are live with positive CLV.
- At least 4 of 6 Phase C features show measurable lift on held-out metrics.
- Per-strategy capital allocator is live with monthly rebalancing.
- Decay-monitoring dashboards show stable or improving edge across all live strategies for 90 consecutive days.
- The system is profitable net of all costs (bridge fees, gas, infrastructure, accounts) over a 6-month window.

14. Open Questions for the User

1. Which external books are accessible from your jurisdiction? Pinnacle availability is the single biggest input to A1's viability.
2. Twitter/X API access tier — Basic (\$100/mo) is enough for ~10k tweets/day; Pro is needed for the curated streaming list at full coverage.

3. Latency budget for B1: is sub-30-second achievable on your infrastructure, or is sub-60-second the realistic target?
4. Risk tolerance for B1 false positives: how many bad bets per month is acceptable in exchange for the edge?
5. Decay-monitoring threshold: at what point do you want to be alerted vs auto-deprecated?
6. Capital scaling plan: at what bankroll do per-strategy caps need to widen, and which strategies absorb additional capital best?